

Dynamic Web Pages with Selenium

Week 8 · Documents module · Textbook Chapter 8

Brian C. Keegan, Ph.D.

Associate Professor, Department of Information Science
University of Colorado Boulder

October 5, 7 & 9, 2026

This week at a glance

Most of the modern web renders its data with JavaScript *after* the HTML arrives. This week: check whether you need a browser, then drive one.

Monday | Concepts & notebook intro

- Three checks by hand; Selenium and Selenium Manager; Playwright scripts; AI agents that drive a browser
- Close with a pair activity: do you need a browser?

Wednesday | Notebook lab

- Work the Chapter 8 notebook in Selenium: xkcd, a Wikipedia search, and the Quotes to Scrape sandbox

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 8

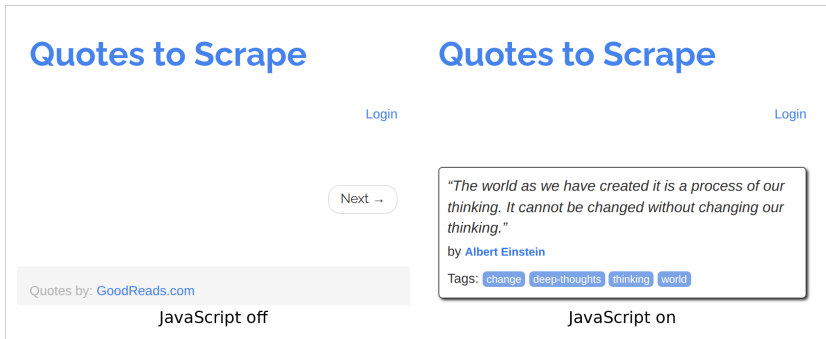
Companion notebook

`ch-08-dynamic-pages.ipynb`

Bring a laptop

Monday and Wednesday are both hands-on — bring a laptop both days.

1. Monday • Concepts



Same URL, two very different pages: `quotes.toscrape.com/js/`, JavaScript off and on.

Week 6 ended here: `requests.get()` with status 202 and zero bytes of HTML. A browser showed the whole page.

Most modern sites are **dynamic**. JavaScript builds the content *after* the HTML arrives, and `requests` never runs JavaScript.

Check by hand first

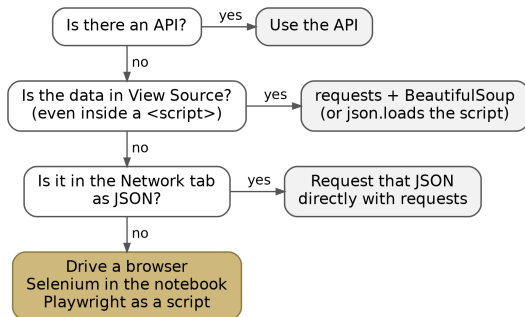
Driving a browser is slow and fragile. Three checks in your own browser settle most pages, about a minute each.

View Source. Is the data in what the server sent?

JavaScript off. What is left without it?

Network tab. Does the data arrive as JSON?

The examples use Quotes to Scrape, a sandbox built for scraping practice.



Check 1: View Source

```
28 <script src="/static/jquery.js"></script>
29 <script>
30   var data = [
31     {
32       "tags": [
33         "change",
34         "deep-thoughts",
35         "thinking",
36         "world"
37       ],
38       "author": {
39         "name": "Albert Einstein",
40         "goodreads_link": "/author/show/9810.Albert_Einstein",
41         "slug": "Albert-Einstein"
42       },
43       "text": "\u201cThe world as we have created it is a process of our thinking. It cannot be changed without changing our thinking.\u201d"
```

`requests` finds zero `div.quote` elements on `quotes.toscrape.com/js/`. View Source shows why: there is no quote markup at all.

But the quotes are in the file — inside a `<script>`, as a JavaScript array. `re.search()` cuts it out and `json.loads()` parses it. Ten quotes, no browser.

Check 2: JavaScript off

Open DevTools. Press Ctrl+Shift+P (Cmd+Shift+P on a Mac). Type `javascript`. Choose **Disable JavaScript**. Reload the page.

What is left is roughly what `requests` gets: a title, a login link, a Next button. No quotes.

JavaScript stays off only while DevTools is open.

Quotes to Scrape

[Login](#)

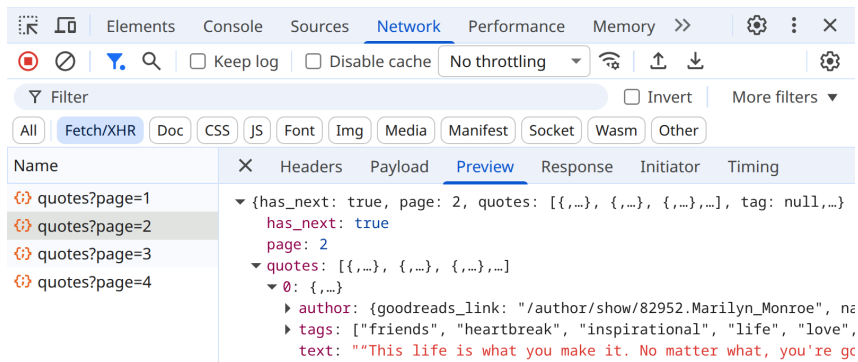
Next →

Quotes by: [GoodReads.com](#)

Made with ♥ by [Zyte](#)

The same page, JavaScript off.

Check 3: the Network tab



On `quotes.toscrape.com/scroll`, filter the Network tab to **Fetch/XHR** and scroll. Each page of quotes arrives as JSON from `/api/quotes?page=N`.

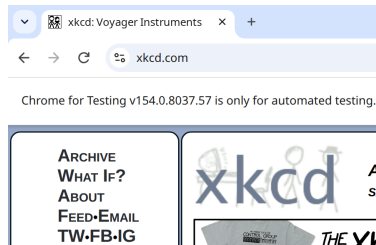
Request that JSON yourself: 100 quotes over 10 pages, with `requests` and a one-second pause between pages. No browser.

Selenium: a browser you drive with code

When all three checks come up empty, you need a real browser.

Selenium is already in `webdata: driver = webdriver.Chrome()` opens a window, and every command you give `driver` happens in that window.

You are not faking HTTP requests anymore. You are **automating a browser**, the same one a person would use.



Chrome for Testing, driven by Selenium.

A small program inside the `selenium` package.

`webdriver.Chrome()` runs it first.

It finds your Chrome, or downloads Chrome for Testing if you have none. It finds the matching driver, downloads it, and caches both in `~/.cache/selenium`.

It also sends anonymous usage statistics, unless you set `SE_AVOID_STATS=true`.

```
chrome not found in the system
Required browser: chrome 154.0.8037.57
Downloading chrome 154.0.8037.57 from https://storage.googleapis.com/...
Required driver: chromedriver 154.0.8037.57
```



When it doesn't

An old chromedriver on your PATH beats the one Selenium Manager picks:

```
Found chromedriver 147.0.7727.24 in PATH: /opt/node22/bin/chromedriver
Driver path: /opt/node22/bin/chromedriver
SessionNotCreatedException: session not created: This version of
ChromeDriver only supports Chrome version 147
Current browser version is 154.0.8037.57
```

Stray driver? Delete it, or set `SE_SKIP_DRIVER_IN_PATH=true`

Campus firewall? Set `SE_PROXY`, or run once off campus

Browser from conda or snap? Set `options.binary_location`

Stuck cache? Delete `~/.cache/selenium`

Set it in Python

```
os.environ["SE_SKIP_DRIVER_IN_PATH"]
= "true" before your first
webdriver.Chrome()
```

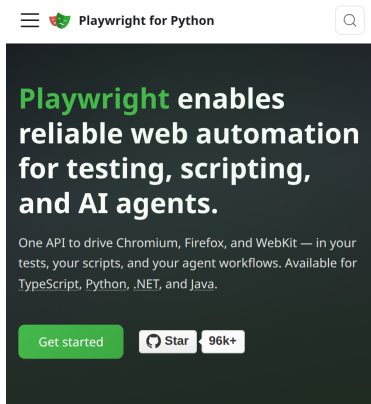
Playwright

Microsoft, January 2020, built by engineers who had worked on Google's Puppeteer.

It brings its own browsers. The library is in `webdata`;
`playwright install chromium` downloads the browsers,
about 300 MB.

It waits for elements on its own, and it records a script while
you click.

It runs best as a script from the terminal, not in a notebook.



The screenshot shows the GitHub repository page for Playwright for Python. At the top, there is a navigation bar with the Playwright logo (a green shield with a white 'P') and the text 'Playwright for Python'. To the right of the navigation bar is a search icon. The main content area has a dark background. It features the headline 'Playwright enables reliable web automation for testing, scripting, and AI agents.' in white and green text. Below the headline, there is a paragraph: 'One API to drive Chromium, Firefox, and WebKit — in your tests, your scripts, and your agent workflows. Available for TypeScript, Python, .NET, and Java.' At the bottom of the main content area, there are two buttons: a green 'Get started' button and a white 'Star' button with a GitHub star icon and the text '96k+'.

playwright.dev/python, September 2026.

A Playwright script

Save it as `xkcd_alt.py`. Run `python xkcd_alt.py` in a terminal.

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch() # headless by default
    page = browser.new_page()
    page.goto("https://xkcd.com")
    img = page.locator("#comic img")
    print(img.get_attribute("alt")) # the comic's name
    print(img.get_attribute("title")) # the hover joke
    browser.close()
```

`page` plays `driver`. `page.goto()` plays `driver.get()`. A `locator` plays `find_element()`. The `with` block is your `driver.quit()`.

This sandbox page waits ten seconds before its JavaScript adds the quotes:

```
page.goto("https://quotes.toscrape.com/js-delayed/")

page.locator("div.quote").count() # 0 -- does not wait
page.locator("div.quote span.text").first.inner_text() # waits ~10 s
page.locator("div.quote").count() # 10
```

Acting on an element, or reading from it, waits up to 30 seconds. Reporting on the page as it stands right now does not.

Selenium needs `WebDriverWait` for the same page. You will write it on Wednesday.

Why not in the notebook?

Run that script in a Jupyter cell:

```
Error: It looks like you are using  
Playwright Sync API inside the asyncio  
loop. Please use the Async API instead.
```

Jupyter already runs an event loop. Playwright's simple sync API needs one of its own.

The async API works in a notebook, with `await` in front of nearly every call. Forget one, and you get a placeholder instead of a result.



Record, then read

playwright codegen

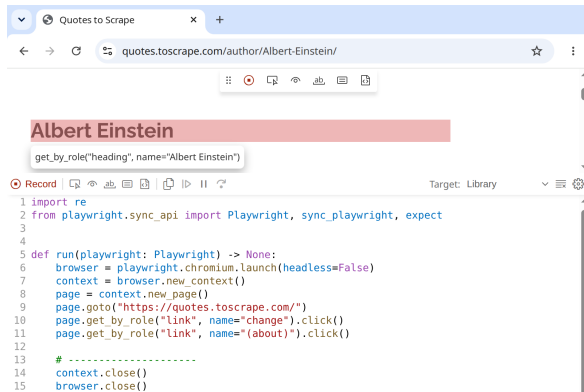
<https://quotes.toscrape.com/>

Click in the browser. Each click becomes a line of Python:

```
page.get_by_role("link",  
    name="change").click()  
page.get_by_role("link",  
    name="(about)").click()
```

Role-based locators survive a CSS redesign.
They break when the visible text changes.

A recording is a first draft. It clicks; it extracts nothing.



MCP, the Model Context Protocol, plugs outside tools into an AI assistant. **Playwright MCP** gives it a browser: `browser_navigate`, `browser_click`, `browser_type`.

These lines work in most MCP-capable assistants (Node.js 20 or newer):

```
{ "mcpServers": {  
  "playwright": {  
    "command": "npx",  
    "args": ["@playwright/mcp@latest"] } } }
```

It opens a visible browser by default. Watch it.



What the agent sees

Not pixels. An **accessibility snapshot**: the page's parts by role and name, the same tree a screen reader uses.

Real links, labeled buttons, and `alt` text make a page easy for an agent. Unlabeled `<div>`s make it hard.

xkcd, September 2026, from Playwright's `aria_snapshot()`.

```
- text: Stargazing 5
- list:
  - listitem:
    - link "<":
      - /url: /1/
    - listitem:
      - link "< Prev":
        - /url: /3300/
      - listitem:
        - link "Random":
          - /url: //c.xkcd.com/random/comic/
        ...
  - img "Stargazing 5"
```

Drive, or write the scraper?

Let it drive. “Collect every quote tagged *love* on quotes.toscrape.com.” Flexible. But every run improvises, costs money, and can click something different.

Let it write. “Write me a Playwright script that collects every quote tagged *love*.” You get code to read, test, commit, and rerun for free.

Codegen. No AI at all. Your own clicks become the script.

Which one could you describe in a methods section?

An agent is still your scraper

Its requests are your requests. `robots.txt` and the Terms of Service still apply (Ch. 2).

The page goes to the AI company along with your prompt. Keep agents away from private data.

Prompt injection: text on a page, even text you cannot see, can give the agent new orders.

Never let an agent drive a browser that is logged in to your accounts. It acts as you.

Automation is not a loophole

A `robots.txt` rule and a site's ToS still apply when a real browser — or an AI — does the clicking.

Not legal advice.

Selenium or Playwright?

	Selenium	Playwright
Install	In <code>webdata</code> (week 1)	In <code>webdata</code> , then <code>playwright install chromium</code>
Browsers	Yours; Selenium Manager downloads Chrome for Testing if you have none	Its own copies, pinned to each release
Notebook	Yes	Only with the async API and <code>await</code>
Waiting	Explicit: <code>WebDriverWait</code>	Automatic when you act on or read an element
Recording	Selenium IDE, a browser extension	<code>playwright codegen</code> , built in

This course's lab and take-home use Selenium.

Closing activity: do you need a browser?

Work in pairs. Pick a page that shows data you would want: a news live blog, a sports scoreboard, a store's search results, a campus events calendar.

1. Open View Source. Search it for a word you can see on the page
2. Turn JavaScript off. Reload. Write down what is left
3. Open the Network tab. Filter to Fetch/XHR. Reload, scroll, or click. Find the request that carries the data
4. Decide: `requests`, the JSON directly, or a browser?

Stuck? Start with quotes.toscrape.com/js/ and quotes.toscrape.com/scroll

Report back with one page that did not need a browser, and one that did



Before Wednesday

Complete daily note: <https://bit.ly/info4617f26>

Read Textbook Ch. 8 before Wednesday.

Made `webdata` before October 2? Do the setup handout's catch-up step, which adds Selenium. Then run `webdriver.Chrome()` once: it may download a driver, and Chrome too if you have none. Do it on a fast connection.

Wednesday lab

Complete **Recommended Exercises**: fill in the empty cells, run everything, export to HTML, upload to Canvas by **Sunday 11:59pm**

Friday

Upload the link to your pull request or issue by **Sunday 11:59pm**

2. Wednesday · Notebook Lab

Launch a driver, then locate elements

Four strategies find elements; `find_element` returns the first match, `find_elements` returns a list:

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome() # Selenium Manager fetches the driver
driver.get("https://xkcd.com")

comic = driver.find_element(By.ID, "comic") # by id
nav = driver.find_elements(By.CSS_SELECTOR, "ul.comicNav li a") # -> a list
title = driver.find_element(By.XPATH, "//div[@id='ctitle']") # by XPath
print(title.text) # the title of the current comic
```

Prefer a stable anchor — an id or a semantic class — over a brittle, position-based path.

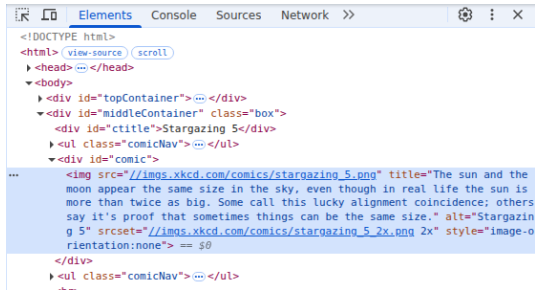
Read attributes, then act like a user

```
# xkcd hides a joke in the image's title (hover) attribute
img = driver.find_element(By.XPATH, "//div[@id='comic']//img")
print(img.get_attribute("alt")) # the comic's name
print(img.get_attribute("title")) # the mouseover punchline

# Anything a user can do, Selenium can do -- e.g. click "Random":
btn = driver.find_element(
    By.XPATH, "//ul[@class='comicNav']//a[contains(text(),'Random')]")
btn.click()
```

`get_attribute()` reads *any* HTML attribute, even ones you never see; `.text` reads the visible text.

Right-click the comic and choose **Inspect** to find both before you write a line.



Type, submit — then wait *well*

Fill a search box, submit, and wait for the exact condition instead of guessing a duration:

```
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver.get("https://www.wikipedia.org")
box = driver.find_element(By.NAME, "search")
box.send_keys("Colorado Buffaloes") # type like a user
box.send_keys(Keys.RETURN) # submit the form

WebDriverWait(driver, 10).until( # not a blind time.sleep(2)!
    EC.presence_of_element_located((By.ID, "firstHeading")))
print(driver.title) # -> Colorado Buffaloes - Wikipedia
```

`WebDriverWait` polls every 0.5 s, returns *the instant* the condition holds, and raises `TimeoutException` if it never does — faster, and it fails loudly. Common conditions: `presence_of_element_located`, `visibility_of_element_located`, `element_to_be_clickable`.

Ten seconds of nothing

The same delayed page from Monday, in Selenium:

```
driver.get("https://quotes.toscrape.com/js-delayed/")

element = WebDriverWait(driver, 15).until( # 15 > the 10-second delay
    EC.presence_of_element_located((By.CLASS_NAME, "quote")))
print(len(driver.find_elements(By.CLASS_NAME, "quote"))) # 10
```

A `time.sleep(2)` looks too early and finds nothing. `WebDriverWait` returns the moment the first quote appears — about ten seconds here.

Set the timeout longer than the slowest page you expect, and no longer than you can stand.

Scraping an infinite-scroll page

Scroll to the bottom, wait, and stop when the page stops growing:

```
def scrape_infinite_scroll(driver, max_scrolls=20, pause=2):
    last = driver.execute_script("return document.body.scrollHeight")
    for i in range(max_scrolls):
        driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
        time.sleep(pause) # let new content load
        new = driver.execute_script("return document.body.scrollHeight")
        if new == last: # height stopped growing -> the end
            break
        last = new
    return driver.page_source # the fully-loaded HTML
```

On `quotes.toscrape.com/scroll` it stops after ten scrolls with 100 quotes. Monday's Network tab check found the same 100 as JSON, with no browser.

Set a sane `max_scrolls`. **Proportionality** (Ch. 2): collect only what your question needs.

Selenium renders; BeautifulSoup reads

Get the page into the state you need, hand the rendered HTML to BeautifulSoup, and **always** clean up:

```
from bs4 import BeautifulSoup

try:
    driver.get("https://quotes.toscrape.com/js/")
    WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.CLASS_NAME, "quote")))
    soup = BeautifulSoup(driver.page_source, "html.parser") # rendered HTML
    for item in soup.find_all("div", class_="quote"):
        text = item.find("span", class_="text")
        if text is None: # same defensive habit as static scraping
            continue # skip a malformed item instead of crashing
        print(text.get_text(strip=True))
finally:
    driver.quit() # each driver is a whole browser process
```

Selenium is the **hands** that navigate; BeautifulSoup is the **eyes** that read. try/finally guarantees the browser closes even when parsing throws.

Three errors you will meet

`NoSuchElementException` — the page has not finished loading. Wait on a condition.

`StaleElementReferenceException` — the page changed after you found the element. Find it again.

`SessionNotCreatedException` — the driver and the browser do not match. Remove the stray driver, or set `SE_SKIP_DRIVER_IN_PATH=true`.

Error flavors

Some failures *are* the lesson (a page that is empty without JavaScript) — that does not need a GitHub issue. Other code may be *genuinely* broken: a selector a site renamed, a sandbox page that moved. Confirm it, then search and file an issue.

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the notebook:

1. Automate a search that has more than one step. Navigate to the page, enter the query, submit it, and extract the result.
2. Examine three sites. Categorize each site as static, partially dynamic, or fully dynamic.
3. Find a “Load More” button. Click it in a loop until it disappears. Count the items that were hidden.
4. Measure the time for `requests`, for Selenium with a window, and for Selenium with no window. Report when the slower method is acceptable.
5. Optional: rewrite your scraper as a Playwright script. Run it from a terminal.
6. **5617:** use both methods on a research site that depends on JavaScript. Write a memo of about 500 words. Refer to Mitchell (2018)

Submitting

Start in class, finish at home, submit what you got working.

Graded on good-faith attempt to complete rather than getting everything working.

Show-and-Tell

Bring one of these to class: a bug you could not solve, data you find interesting, or a question for a research design.

The Twitter scraper that worked in 2019 was dead by 2024.

Browser automation is borrowed time — write it defensively, and expect it to break.

3. Friday • Textbook Revisions

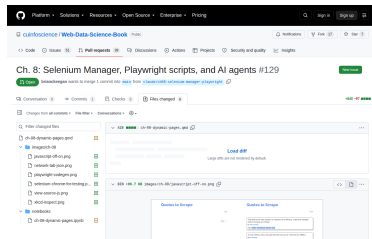
Improving Chapter 8 together

Today we make the book better. Each of you opens a **pull request** against `ch-08-dynamic-pages.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

Review comments still follow Week 3's claim / link / sharpen taxonomy — claim what you'll fix, link related issues, sharpen ones missing detail.



High-value targets — pick one and scope it to a single PR:

- **Test the code against live sites.** xkcd, Wikipedia, and the Quotes to Scrape sandbox all drift. Run the notebook headless, report what broke, and fix the selector — the chapter's own “Fragility Problem.”
- **Test the Playwright scripts.** Run the chapter's scripts from a terminal on your own machine. Report the download size and anything that differed on Windows or macOS.
- **Add the Firefox path.** Show Selenium Manager fetching `geckodriver`, and `playwright install firefox`, with what changed.
- **Try an agent on the sandbox.** Have an AI agent write a scraper for `quotes.toscrape.com`. Compare its script with yours, and add what you learned to “Letting an AI Agent Drive.”
- **Strengthen an exercise.** Add expected output or a starter cell so an Additional Exercise is self-checking.
- **Extend “Further Reading.”** A public-interest study that turned to browser automation after an API closed.

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates “must fix” from “nice to have”
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Extracting data from PDFs** — when the data isn't on a web page at all.

Key takeaways

1. Check by hand first: View Source, JavaScript off, the Network tab. The data is often already there.
2. Selenium drives a **real browser** from your notebook. Selenium Manager fetches the driver, and the browser if you need one.
3. Wait on **conditions**, not a blind `time.sleep()`. Render with a browser, read with BeautifulSoup, and `driver.quit()` every time.
4. Playwright scripts the same work from a terminal. It waits on its own and records while you click.
5. Agents can drive the browser, but for research, have them write a script you can rerun. `robots.txt` and ToS still apply.